

CSE110A: Compilers

Topics:

- *Optimizations: Constant Folding*

Constant propagation and constant folding

- Colloquially, they are often used interchangeably
- Technically (e.g. according to the books)
 - Constant propagation is replacing variables with constants
 - Constant folding is compile-time evaluation when constants are known

Constant propagation and constant folding

```
int x = 14;  
int y = 7 - x / 2;  
return y * (28 / x + 2);
```

Constant propagation and constant folding

```
int x = 14;  
int y = 7 - x / 2;  
return y * (28 / x + 2);
```

constant propagation

```
int x = 14;  
int y = 7 - 14 / 2;  
return y * (28 / 14 + 2);
```

Constant propagation and constant folding

```
int x = 14;  
int y = 7 - x / 2;  
return y * (28 / x + 2);
```

constant propagation

```
int x = 14;  
int y = 7 - 14 / 2;  
return y * (28 / 14 + 2);
```

*constant
folding*

```
int x = 14;  
int y = 0;  
return y * (4);
```

Constant propagation and constant folding

```
int x = 14;  
int y = 7 - x / 2;  
return y * (28 / x + 2);
```

constant propagation

```
int x = 14;  
int y = 7 - 14 / 2;  
return y * (28 / 14 + 2);
```

*constant
folding*

```
int x = 14;  
int y = 0;  
return 0 * (4);
```

*constant
propagation*

```
int x = 14;  
int y = 0;  
return y * (4);
```

Constant propagation and constant folding

```
int x = 14;  
int y = 7 - x / 2;  
return y * (28 / x + 2);
```

constant propagation

```
int x = 14;  
int y = 7 - 14 / 2;  
return y * (28 / 14 + 2);
```

*constant
folding*

```
int x = 14;  
int y = 0;  
return 0 * (4);
```

*constant
propagation*

```
int x = 14;  
int y = 0;  
return y * (4);
```

*constant
folding*

```
int x = 14;  
int y = 0;  
return 0;
```

Typically performed at the same time

```
int x = 14;  
int y = 7 - x / 2;  
return y * (28 / x + 2);
```

constant propagation and folding second line

```
int x = 14;  
int y = 0;  
return y * (28 / x + 2);
```

```
int x = 14;  
int y = 0;  
return 0;
```

constant propagation and folding third line

Adding constant folding to LVN

```
b = 5;  
c = 3;  
  
a = b + c;  
b = a - d;  
c = a + c;  
d = a - d;
```

```
H = {  
}
```

```
Known_values = {  
  
}
```

Adding constant folding to LVN

numbering

```
b0 = 5;  
c1 = 3;  
  
a2 = b0 + c1;  
b4 = a2 - d3;  
c5 = a2 + c1;  
d6 = a2 - d3;
```

```
H = {  
}
```

```
Known_values = {  
  
}
```

Adding constant folding to LVN

As you are iterating through code, add any constant mappings to Known_values:

```
b0 = 5;  
c1 = 3;  
  
a2 = b0 + c1;  
b4 = a2 - d3;  
c5 = a2 + c1;  
d6 = a2 - d3;
```

```
H = {  
}
```

```
Known_values = {  
  
}
```

Adding constant folding to LVN

As you are iterating through code, add any constant mappings to Known_values:

```
b0 = 5;
```

```
c1 = 3;
```

```
a2 = b0 + c1;
```

```
b4 = a2 - d3;
```

```
c5 = a2 + c1;
```

```
d6 = a2 - d3;
```

```
H = {  
}
```

```
Known_values = {  
  "b0" : 5  
  "c1" : 3  
  
}
```

Adding constant folding to LVN

When you find an arithmetic operation, first check if operands are known

```
b0 = 5;  
c1 = 3;  
a2 = b0 + c1;  
b4 = a2 - d3;  
c5 = a2 + c1;  
d6 = a2 - d3;
```

```
H = {  
}
```

```
Known_values = {  
  "b0" : 5  
  "c1" : 3  
  
}
```

Adding constant folding to LVN

When you find an arithmetic operation, first check if operands are known

```
b0 = 5;  
c1 = 3;  
a2 = b0 + c1;  
b4 = a2 - d3;  
c5 = a2 + c1;  
d6 = a2 - d3;
```

5 + 3

evaluate and add to known values

H = {
}

Known_values = {
 "b0" : 5
 "c1" : 3
}

Adding constant folding to LVN

When you find an arithmetic operation, first check if operands are known

```
b0 = 5;  
c1 = 3;  
a2 = 8;  
b4 = a2 - d3;  
c5 = a2 + c1;  
d6 = a2 - d3;
```

5 + 3

evaluate and add to known values

H = {
}

Known_values = {
 "b0" : 5
 "c1" : 3
 "a2" : 8
}

Adding constant folding to LVN

When you find an arithmetic operation, first check if operands are known

```
b0 = 5;  
c1 = 3;  
  
a2 = 8;  
b4 = 8 - d3;  
c5 = a2 + c1;  
d6 = a2 - d3;
```

propagate constant (if IR allows it)

```
H = {  
  
}
```

```
Known_values = {  
  "b0" : 5  
  "c1" : 3  
  "a2" : 8  
  
}
```


Adding constant folding to LVN

When you find an arithmetic operation, first check if operands are known

```
b0 = 5;  
c1 = 3;  
  
a2 = 8;  
b4 = 8 - d3;  
c5 = a2 + c1;  
d6 = a2 - d3;
```

add to H

why do we want to store 8
here rather than a2?

```
H = {  
    "8 - d3" : "b4"  
}
```

```
Known_values = {  
    "b0" : 5  
    "c1" : 3  
    "a2" : 8  
  
}
```

continue on.

Arithmetic identities

```
b0 = 0;  
d3 = 1;  
f7 = 4;  
  
a2 = b0 + c1;  
b4 = a2 * d3;  
d6 = e5 * f7;
```

```
H = {  
}
```

```
Known_values = {  
}
```

Arithmetic identities

what can we do here?

```
b0 = 0;  
d3 = 1;  
f7 = 4;  
  
a2 = b0 + c1;  
b4 = a2 * d3;  
d6 = e5 * f7;
```

```
H = {  
}
```

```
Known_values = {  
  "b0":0, "d3":1, "f7":4  
}
```

Arithmetic identities

what can we do here?
add a special rule for +
that if any side is 0, you can
just drop the 0.

```
b0 = 0;  
d3 = 1;  
f7 = 4;  
  
a2 = b0 + c1;  
b4 = a2 * d3;  
d6 = e5 * f7;
```

```
H = {  
}
```

```
Known_values = {  
  "b0":0, "d3":1, "f7":4  
}
```

Arithmetic identities

what can we do here?
add a special rule for +
that if any side is 0, you can
just drop the 0.

```
b0 = 0;  
d3 = 1;  
f7 = 4;  
  
a2 = c1;  
b4 = a2 * d3;  
d6 = e5 * f7;
```

```
H = {  
}
```

```
Known_values = {  
  "b0":0, "d3":1, "f7":4  
}
```

What other rules could we have?